



UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 7

DC MOTOR SPEED MEASUREMENT

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 7

DC Motor Speed Measurement System

Objectives

1. To understand the methods of acquiring measurements automatically with the aid of a single-board computer.
2. To become familiar with GPIO-based data acquisition on the Raspberry Pi.
3. To become familiar with signal conditioning in a computer-based data-acquisition system.
4. To design and build a signal-conditioning circuit whose parameters are justified by a measurement, rather than assumed.
5. To implement a computer-based measurement system for the rotational speed of a DC motor, using a Raspberry Pi in place of a PC-hosted DAQ card.

Equipment / Software Required

- DC motor trainer with a 12-slot wheel encoder
- Digital storage oscilloscope (DSO)
- Op-amps, resistors and capacitors
- Raspberry Pi 4B
- Breadboard and jumper wires (male–female, for Pi GPIO header connections)
- Multimeter

Introduction

In most measurement processes it is often necessary to acquire data automatically, without a human operator. A single-board computer such as the Raspberry Pi is well suited to this: it has several advantages over conventional measuring instruments and over a PC-hosted DAQ card, including a built-in GPIO header for direct digital interfacing, low cost, and the ability to run continuously as a dedicated instrument.

A computer-based measurement system typically contains the following elements (Figure 7.1): a transducer, a signal-conditioning stage, a digital interface, and software running on the Raspberry Pi itself, rather than a traditional PC.

Signal conditioning means manipulating an analogue signal so that it meets the requirements of the next processing stage — most commonly, so that it is suitable for a digital input. Signal conditioning can include amplification, filtering, range matching, and isolation.

Filtering is the most common signal-conditioning function: frequencies that do not carry valid data are attenuated. In this experiment the encoder produces a pulse train whose useful frequency content is bounded, and any filter used ahead of the Pi's GPIO input should be designed — not assumed — around that bound.

Amplification increases the resolution of the input signal and improves its signal-to-noise ratio.

Level matching is critical when interfacing to a Raspberry Pi: its GPIO pins are **3.3 V logic and are not 5 V-tolerant**. Any conditioning circuit driving a Pi GPIO input must guarantee its output never exceeds 3.3 V, and should include a clean logic-level transition (e.g. via a comparator or Schmitt-trigger stage) rather than a slow analogue ramp, so that GPIO edge-detection triggers reliably.

The Sallen–Key topology is a widely used active-filter structure, valued for its simplicity: it realises a two-pole (12 dB/octave) low-pass, high-pass, or band-pass response using a single op-amp configured as a near-unity-gain voltage-controlled voltage source (VCVS), together with two resistors and two capacitors that set the cutoff frequency and Q .

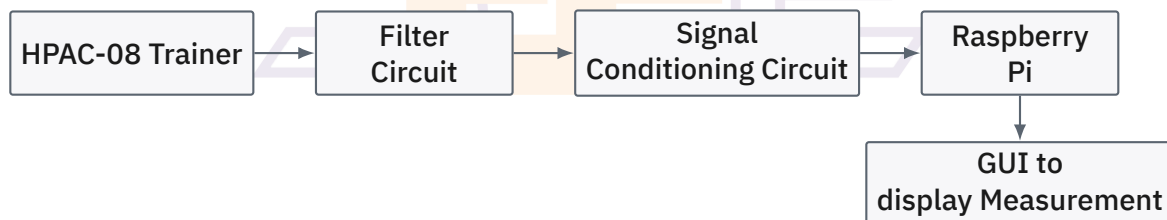


Figure 7.1: Computer-based measurement system.

Procedure

Activity 1 — Characterising the encoder signal and designing the filter

Objective: measure the encoder's pulse characteristics and design a signal-conditioning filter around the measured cutoff frequency, rather than an assumed one.

1. Wire the encoder's output test point to the oscilloscope.
2. Run the motor across its usable speed range and observe the encoder pulse train on the oscilloscope.
3. At a representative operating speed, measure the pulse period and estimate the corresponding fundamental frequency from the oscilloscope trace.
4. From this measurement, determine a suitable cutoff frequency for the noise-rejection filter.

- Design the necessary filter based on the cutoff frequency determined above. Justify your design choices.
- Design the signal-conditioning circuit needed to present the filtered encoder pulses to a Raspberry Pi GPIO input with valid 3.3 V logic levels (**never exceeding 3.3 V**). Justify your design choices.

Activity 2 — Interfacing to the Raspberry Pi GPIO

Objective: connect the conditioned encoder signal to a Raspberry Pi GPIO input pin.

- Connect the output of your signal-conditioning circuit to a GPIO input pin on the Raspberry Pi header (see Figure 7.2 and the Raspberry Pi GPIO pinout reference).
- Before connecting**, verify with a multimeter that your conditioning circuit's output never exceeds 3.3 V across the motor's full operating range — an over-voltage GPIO input can permanently damage the Raspberry Pi.
- Confirm the expected pulses are being registered using a simple polling test script (Python: `read GPIO.input(pin)` in a tight loop and print transitions; C: poll the pin state via `libgpiod` or `wiringPi`) before writing the full measurement program.

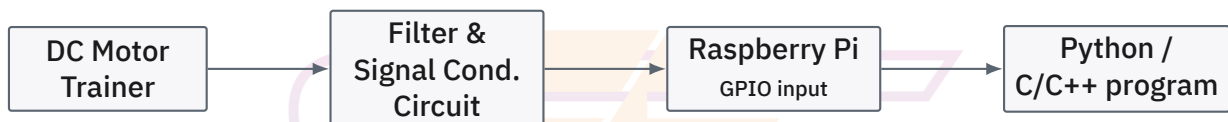


Figure 7.2: Signal path from the DC motor trainer to the Raspberry Pi.

Activity 3 — Speed measurement software

Objective: implement software to compute and display motor speed (frequency and RPM) in real time, using two selectable measurement methods.

Two complementary methods can be used to convert the pulse train into a frequency and rotational speed, and your software should let the user select between them:

Method 1 — by counts (fixed pulse count, 1–99): the user specifies a target pulse count N ($1 \leq N \leq 99$); the software measures the elapsed time T_N for N pulses to occur, then computes

$$f = \frac{N}{T_N \times 12} \text{ [rev/s]}, \quad \text{RPM} = 60f, \quad \omega = 2\pi f \text{ [rad/s]}$$

This method gives better resolution at high speed (many pulses arrive quickly, so T_N is measured over a longer, more precise interval) but responds more slowly at low speed, since a full count of N pulses can take a long time.

Method 2 — by period (fixed period window, 1–9 s): the user specifies a measurement window T seconds ($1 \leq T \leq 9$); the software counts the number of pulses N_T that occur within that window, then computes

$$f = \frac{N_T}{T \times 12} \text{ [rev/s]}, \quad \text{RPM} = 60f, \quad \omega = 2\pi f \text{ [rad/s]}$$

In both methods, t_{start} and t_{stop} (marking the beginning and end of the measurement) are obtained from a monotonic system clock rather than a DAQ card's counter:

$$\text{Elapsed time} = t_{\text{stop}} - t_{\text{start}}$$

GPIO access method matters for timing accuracy. Polling a GPIO pin in a plain software loop (busy-waiting) is simple but imprecise, since Python/Linux scheduling jitter can distort short pulse timings.

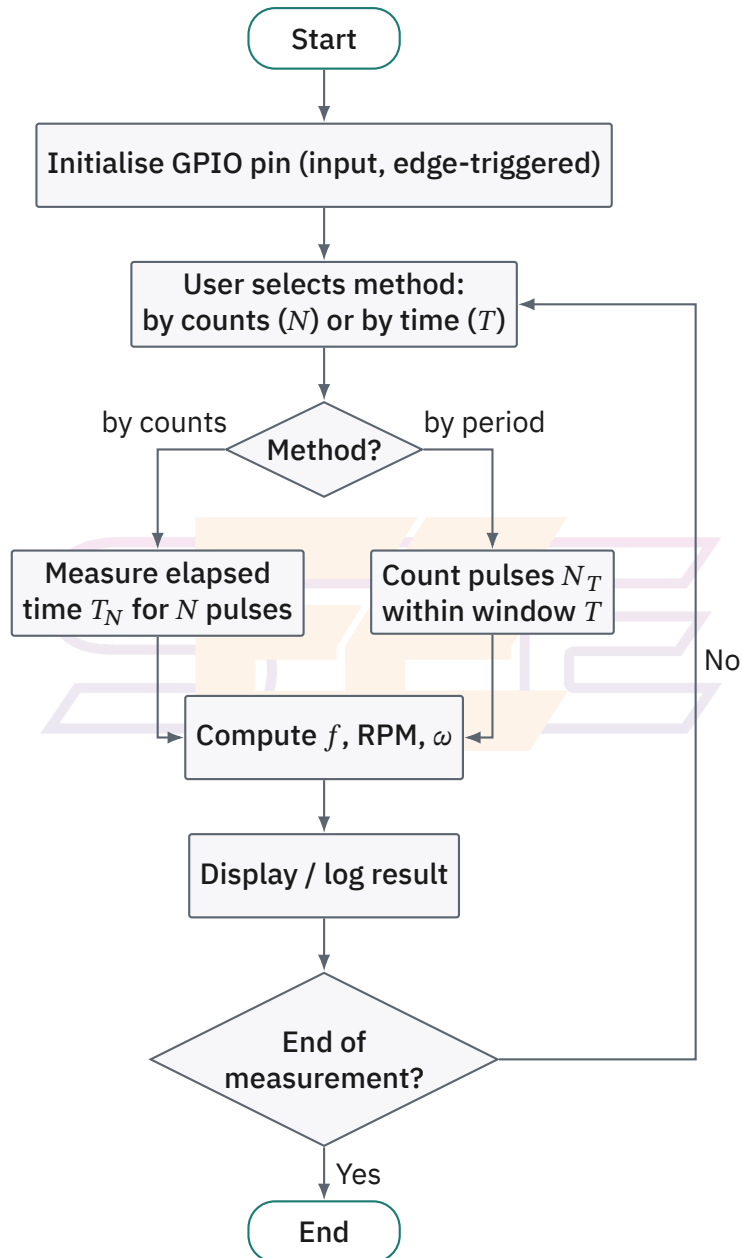


Figure 7.3: Measurement loop for the speed-acquisition software.

1. In Python or C/C++, write a program that:

- initialises the chosen GPIO pin as an edge-triggered input (see library recommendations above),
- lets the user select the measurement method (by counts, $1 \leq N \leq 99$; or by time, $1 \leq T \leq 9$ s) and enter the corresponding parameter,

- for the *by counts* method: times how long it takes for N encoder pulses to occur,
 - for the *by time* method: counts how many encoder pulses occur within the window T ,
 - computes the frequency, RPM, and angular speed ω using the relations above, and
 - displays the running value (frequency and RPM, with the active method indicated) in a simple terminal or GUI display, updating in real time.
2. Interface your filter/conditioning circuit and the Raspberry Pi GPIO input together and perform measurements across a range of motor input voltages, using both methods at each operating point. Compare your computed speed against a direct oscilloscope measurement of the encoder period at the same operating point.
 3. Tabulate your results (Tables 7.2–7.3).
 4. Plot computed speed (from both software methods) against the oscilloscope-measured speed.
 5. Discuss and comment on your results, including any discrepancy between the two software methods and the oscilloscope reference, and how each method's resolution changes with motor speed (see the trade-off noted above).

Table 7.1: By-counts method: frequency comparison table.

N	Measured Frequency (Hz)					Oscilloscope (Hz)
	#1	#2	#3	#4	#5	

Table 7.2: By-counts method: RPM comparison table.

N	ω (Measured RPM)					Oscilloscope (calculated RPM)
	#1	#2	#3	#4	#5	

Table 7.3: By-period method: frequency comparison table.

T (s)	Measured Frequency (Hz)					Oscilloscope (Hz)
	#1	#2	#3	#4	#5	

Table 7.4: By-period method: RPM comparison table.

T (s)	ω (Measured RPM)					Oscilloscope (calculated RPM)
	#1	#2	#3	#4	#5	

Report

1. Present your oscilloscope measurements of the encoder pulse and your resulting filter design (topology, calculations, and component values), together with the measured response of the built filter.
2. Present your design of the signal-conditioning stage between the filter and the Raspberry Pi GPIO input, with justification, including how you ensured the 3.3 V logic-level limit was respected.
3. Compare your software-measured speed (both by-counts and by-time methods) with the oscilloscope-measured speed across your tested operating points, and discuss the sources of any discrepancy.
4. Compare the by-counts and by-time methods directly: at what motor speeds does each method give better resolution or a more stable reading, and why? Which would you recommend for a low-speed application, and which for a high-speed one?
5. What is the theoretical maximum measurable speed of this system, given the encoder resolution (12 slots/revolution) and your software's timing resolution?